Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»

Кафедра 09.02.07_Информационные системы и программирование

ПРАКТИЧЕСКАЯ РАБОТА № 5

Дисциплина(модуль)/предмет МДК 03.01 Внедрение и поддержка компьютерных систем

Тема: Проектирование безопасных релизных процессов, паттернов миграции данных и эксплуатационной документации ИС «Умный кампус»

ВАРИАНТ № 10 Спортивный комплекс (CAMPUS-110: изменение формата TimeSlot → TimeRange)

Студент (ка)/учащийся: Леонов Тимофей Андреевич /_____ /
Ф. И. О. подпись

Группа: И14-23-9

Руководитель: Чудинова А.А /_____ /
Должность, степень, Фамилия и инициалы подпись

Дата представления работы «22» октября 2026г.

Работа аттестована с оценкой _____
прописью

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 ИСХОДНЫЕ ДАННЫЕ ВАРИАНТА.....5
2 ЕДИНЫЙ ЦИФРОВОЙ СЛЕД.....7
3 CHANGE REQUEST НА РЕЛИЗ 1.9.0.....9
3.1 Карточка изменения.....9
3.2 Обоснование окна обслуживания.....10
3.3 План проведения работ.....10
3.4 План отката.....11
4 ПРАВИЛА ЗАЩИТЫ ВЕТКИ MAIN.....12
5 МИГРАЦИЯ БАЗЫ ДАННЫХ ПО ПАТТЕРНУ EXPAND AND CONTRACT.....14
5.1 Общая схема миграции.....14
5.2 Фаза Expand.....15
5.3 Фаза Migrate.....17
5.4 Фаза Contract.....17
5.5 Проверка миграций.....18
6 PRODUCTION SANITY-ТЕСТ.....19
7 ЭКСПЛУАТАЦИОННАЯ ДОКУМЕНТАЦИЯ.....21
7.1 Архитектурное решение ADR-0009.....21
7.2 Журнал изменений.....22
ЗАКЛЮЧЕНИЕ.....23
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....24

ВВЕДЕНИЕ

Модуль «Спортивный комплекс» ИС «Умный кампус» обслуживает бронирование залов, бассейна и кортов студентами и сотрудниками через мобильное приложение, сайт и турникеты. Сервис работает круглосуточно, поэтому любое изменение – новая версия кода или изменение структуры базы данных – должно выпускаться без простоя и с возможностью быстрого отката.

Актуальность темы связана с тем, что в версии 1.8.3 интервал брони хранится в виде строки. Это приводит к двойному бронированию одной площадки, к расхождению времени между клиентами в разных часовых поясах и к невозможности индексировать поиск свободных окон. Исправление требует изменить формат поля в таблице с десятками тысяч записей, которой одновременно пользуются старые и новые экземпляры приложения. Ошибка в такой миграции или неуправляемый релиз приводят к потере броней и остановке сервиса, поэтому нужны формализованный процесс управления изменениями [6, 7] и безопасный паттерн миграции данных [8].

Объект исследования – процесс выпуска релизов сервиса campus-sports-api модуля «Спортивный комплекс». Предмет исследования – методы безопасного релиза, миграции схемы базы данных без простоя и ведения эксплуатационной документации.

Новизна работы состоит в объединении в одну прослеживаемую цепочку задачи Jira, коммита, проверок CI, релиза, трассировки запросов OpenTelemetry и записи о миграции, а также в применении диапазонного типа PostgreSQL с ограничением исключения, которое физически запрещает пересекающиеся брони.

Практическая значимость: подготовленные Change Request, правила защиты ветки, SQL-скрипты миграции, sanity-тест и документация могут

использоваться командой при выпуске версии 1.9.0 и последующих релизов модуля.

Цель работы – спроектировать безопасный релиз изменения формата TimeSlot на TimeRange для модуля «Спортивный комплекс» с миграцией данных без простоя и полным комплектом эксплуатационной документации.

Для достижения цели поставлены задачи:

– добавить в контроллер бронирований единый цифровой след – теги OpenTelemetry с номером задачи и идентификатором брони;

– оформить Change Request на релиз 1.9.0 с обоснованием окна обслуживания и планом отката;

– описать правила защиты ветки main;

– разработать SQL-скрипты миграции по паттерну Expand and Contract и проверить их на PostgreSQL 16;

– написать production sanity-тест с очисткой тестовых данных;

– подготовить архитектурное решение ADR и журнал изменений.

Методы работы: анализ предметной области, моделирование процесса миграции (схема выполнена в draw.io), разработка на C# и SQL, автоматизированное тестирование и непрерывная интеграция в GitHub Actions.

1 ИСХОДНЫЕ ДАННЫЕ ВАРИАНТА

Вариант № 10. Модуль ИС «Умный кампус» – «Спортивный комплекс» (бронирование залов, бассейна, кортов). Задача Jira – CAMPUS-110. Суть миграции базы данных – изменение формата поля TimeSlot со строки на структуру TimeRange.

Исходное состояние (версия 1.8.3): интервал брони хранится в колонке sports.bookings.time_slot типа varchar(32) в виде строки «2026-11-05 18:00-19:30». Пересечение интервалов проверяется строками в коде неатомарно, в базе встречаются два формата строки, а поиск свободных окон нельзя проиндексировать.

Целевое состояние: в C# – объект-значение TimeRange(Start, End), полуоткрытый интервал в UTC; в PostgreSQL – диапазонный тип tstzrange [5] с ограничением исключения EXCLUDE USING gist, которое физически запрещает пересекающиеся активные брони одной площадки.

Все исходные файлы размещены в репозитории https://github.com/kotokotru/smart-campus в папке homework-5. Сервис реализован на ASP.NET Core 8, СУБД – PostgreSQL 16, тесты – xUnit, CI – GitHub Actions. Соответствие блоков задания и файлов приведено в таблице 1.

Таблица 1 – Блоки задания и файлы репозитория

Блок задания	Файл в репозитории
Цифровой след	src/Campus.Sports.Api/Controllers/SportsBookingsController.cs, Telemetry/CampusTelemetry.cs
Управление изменениями	docs/change-requests/CR-2026-110-v1.9.0.md, docs/branch-protection.md
Expand and Contract	db/migrations/01_expand.sql, 02_migrate.sql, 03_contract.sql, Data/NpgsqlBookingRepository.cs
Sanity-тест и Docs-as-Code	tests/Campus.Sports.Tests/ProductionSanityTests.cs, docs/adr/0009-timeslot-to-timerange.md, CHANGELOG.md

Источник: составлено автором

Структура папки homework-5 в репозитории показана на рисунке 1.

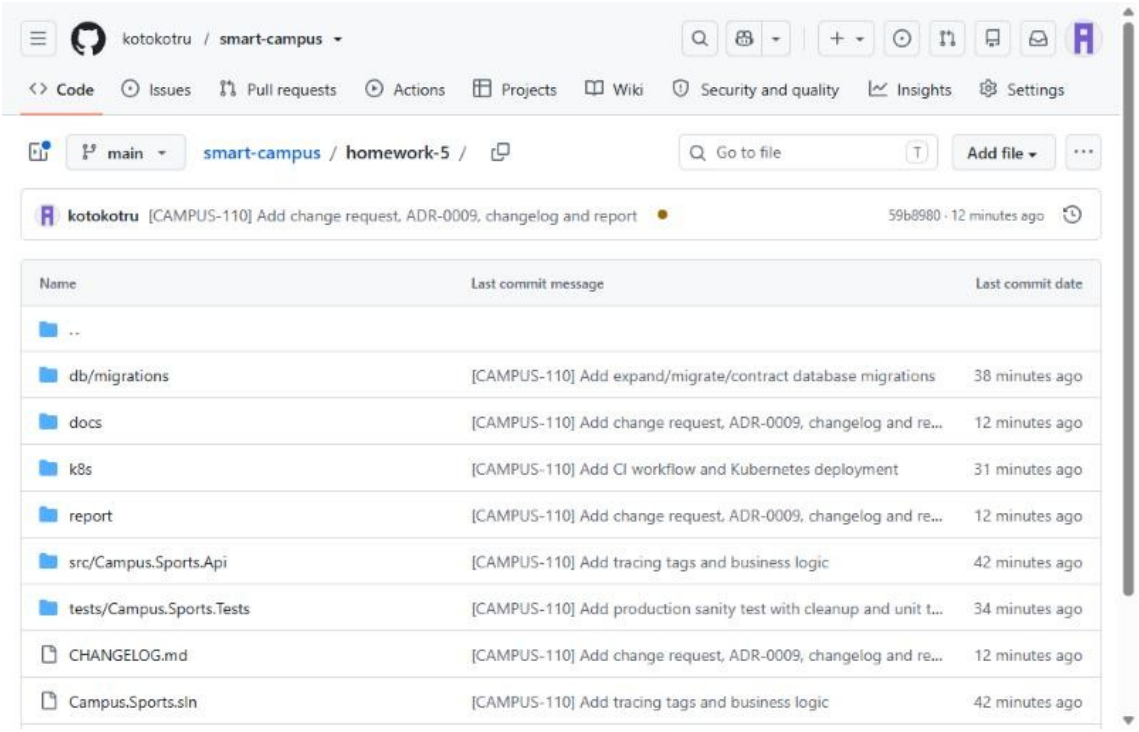


Рисунок 1 – Папка homework-5 в репозитории на GitHub

2 ЕДИНЫЙ ЦИФРОВОЙ СЛЕД

В контроллер SportsBookingsController добавлен проброс цифрового следа OpenTelemetry. Теги устанавливаются в текущий span HTTP-запроса Activity.Current, который создаёт инструментирование ASP.NET Core, и в дочерний доменный span SportsBooking.Create из собственного ActivitySource «Campus.Sports» [1, 2] (рисунок 2).

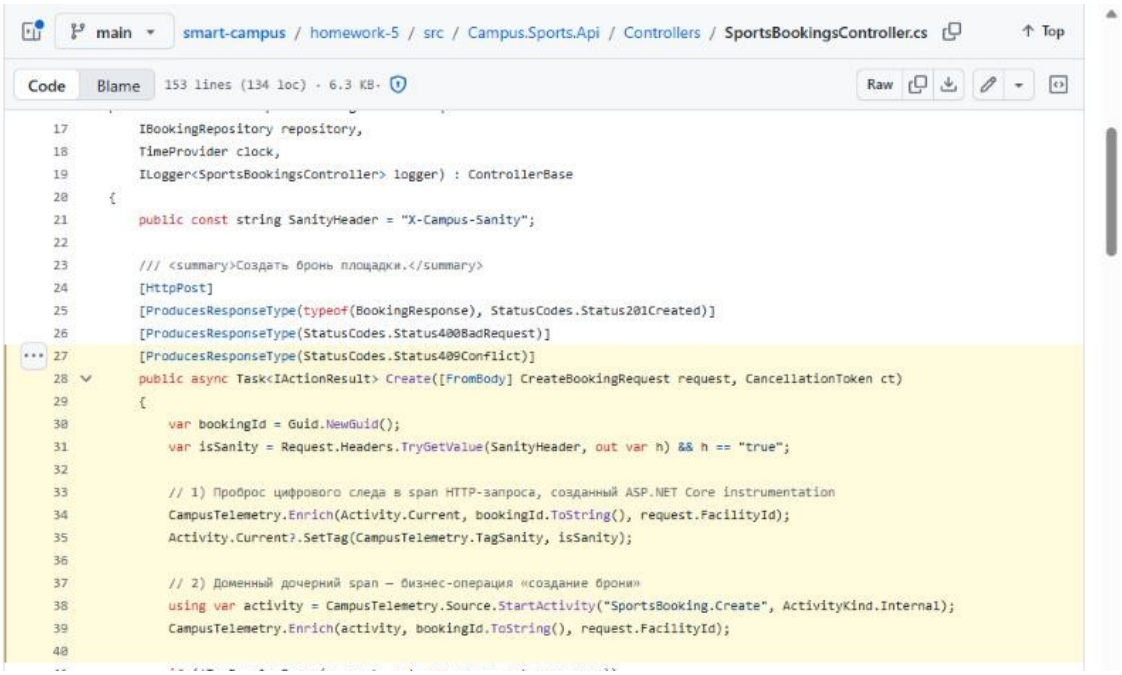


Рисунок 2 – Проброс тегов в Activity.Current в методе Create

Листинг 1 – Установка обязательных тегов (CampusTelemetry.Enrich)

```
public static void Enrich(Activity? activity, string? bookingId = null,
                          string? facilityId = null)
{
   if (activity is null) return;
   activity.SetTag("jira.issue", "CAMPUS-110");
   activity.SetTag("campus.db.migration_phase", "expand");
   if (bookingId  is not null) activity.SetTag("campus.booking_id",  bookingId);
   if (facilityId is not null) activity.SetTag("campus.facility_id", facilityId);
}
```

Обязательные теги – jira.issue = CAMPUS-110 и доменный идентификатор сущности campus.booking_id. Дополнительно передаются campus.facility_id, границы интервала и campus.request.format: по последнему тегу видно, сколько клиентов ещё присылают старый формат, что помогает

принять решение о фазе Contract. В Program.cs источник зарегистрирован в конвейере трассировки вместе с экспортом OTLP, в ресурс добавлены версия сервиса 1.9.0 и хеш коммита.

Корректность установки тегов проверяет тест TracingTagsTests: через ActivityListener он перехватывает завершённые span и проверяет, что и доменный, и родительский HTTP-span содержат jira.issue = CAMPUS-110 и тот же campus.booking_id, что вернул API.

Изменения зафиксированы коммитом с сообщением «[CAMPUS-110] Add tracing tags and business logic» (8fbed04). Формат [CAMPUS-NNN] проверяется в CI (задание commit-lint). В итоге формируется сквозная цепочка цифрового следа: задача Jira, коммит, CI, образ 1.9.0, аннотация развёртывания, span с jira.issue и запись в таблице миграций. История коммитов репозитория приведена на рисунке 3.

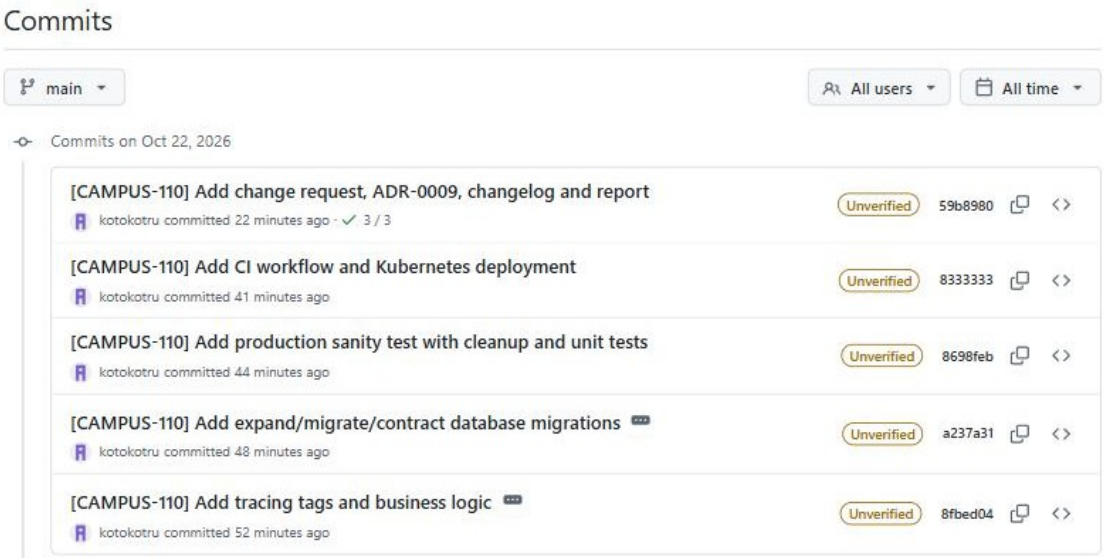


Рисунок 3 – История коммитов, привязанных к задаче CAMPUS-110

3 CHANGE REQUEST НА РЕЛИЗ 1.9.0

3.1 Карточка изменения

Карточка Change Request оформлена в файле docs/change-requests/CR-2026-110-v1.9.0.md по практике управления изменениями ITIL [7]. Основные поля приведены в таблице 2, вид документа в репозитории – на рисунке 4.

Таблица 2 – Карточка CR-2026-110

Поле	Значение
Номер, задача	CR-2026-110, CAMPUS-110
Сервис	campus-sports-api, версия 1.8.3 → 1.9.0
Тип, риск	Normal Change (CAB), риск средний; ожидаемый простой – 0 мин
Описание	ввод TimeRange/tstzrange вместо строки TimeSlot; фазы Expand и Migrate; API принимает и возвращает оба формата; теги OpenTelemetry; sanity-тест
Бизнес-цель	исключить двойное бронирование площадок, единое время брони во всех клиентах, основа для поиска свободных окон
Окно обслуживания	среда 28.10.2026, 03:00–05:00 МСК
Согласующие	тимлид платформы, DBA, владелец продукта, руководитель спорткомплекса

Источник: составлено автором

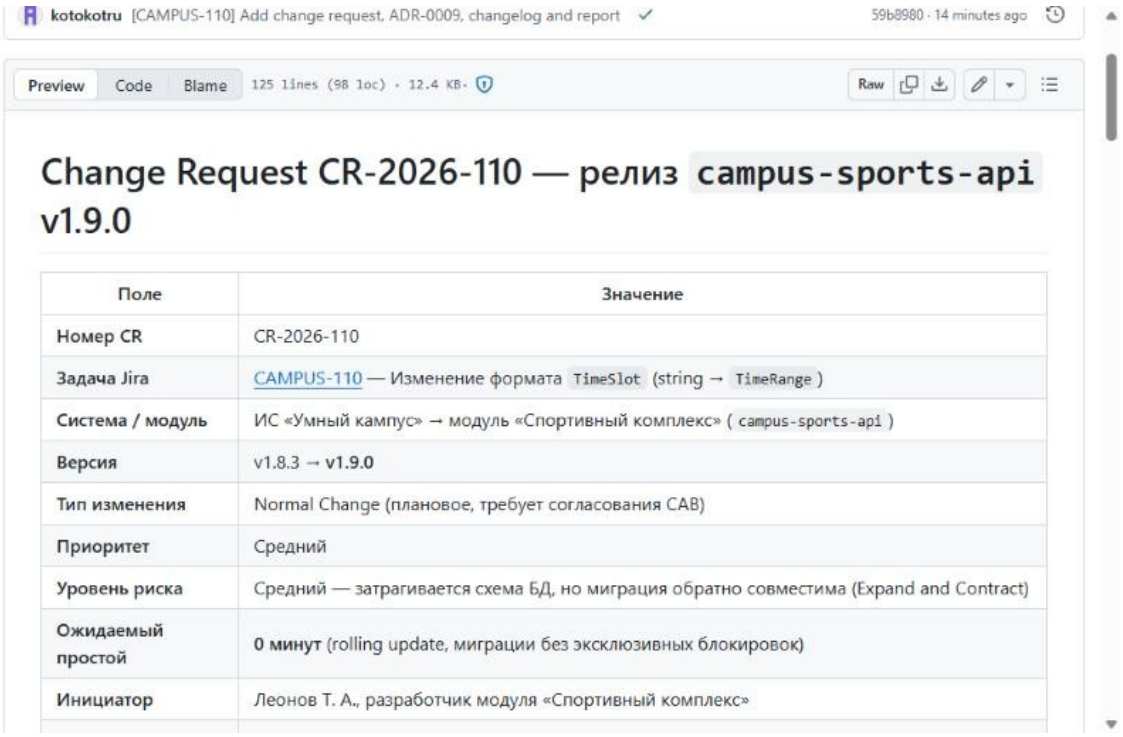


Рисунок 4 – Карточка CR-2026-110 в репозитории

3.2 Обоснование окна обслуживания

По данным мониторинга за восемь недель минимальный трафик сервиса приходится на период с 03:00 до 05:00 (около 2 запросов в секунду против 180 в пике с 17:00 до 21:00). Спорткомплекс закрыт с 23:00 до 07:00, турникеты не работают. Ночное резервное копирование БД завершается в 02:30, поэтому к началу работ есть свежая копия. До открытия в 07:00 остаётся два часа на откат. Среда выбрана потому, что в понедельник в 08:00 открывается запись на неделю, а релиз в пятницу оставил бы возможную проблему на выходные без дежурной команды.

3.3 План проведения работ

Работы выполняются по шагам:

1) 03:00 – фиксация текущей ревизии развёртывания командой kubectl rollout history;

2) 03:05 – фаза Expand: выполнение 01_expand.sql (DBA);

3) 03:10 – выкат образа 1.9.0 командой kubectl set image и запись причины изменения;

4) 03:12 – контроль завершения rolling update (maxUnavailable: 0) [6];

5) 03:20 – запуск production sanity-теста;

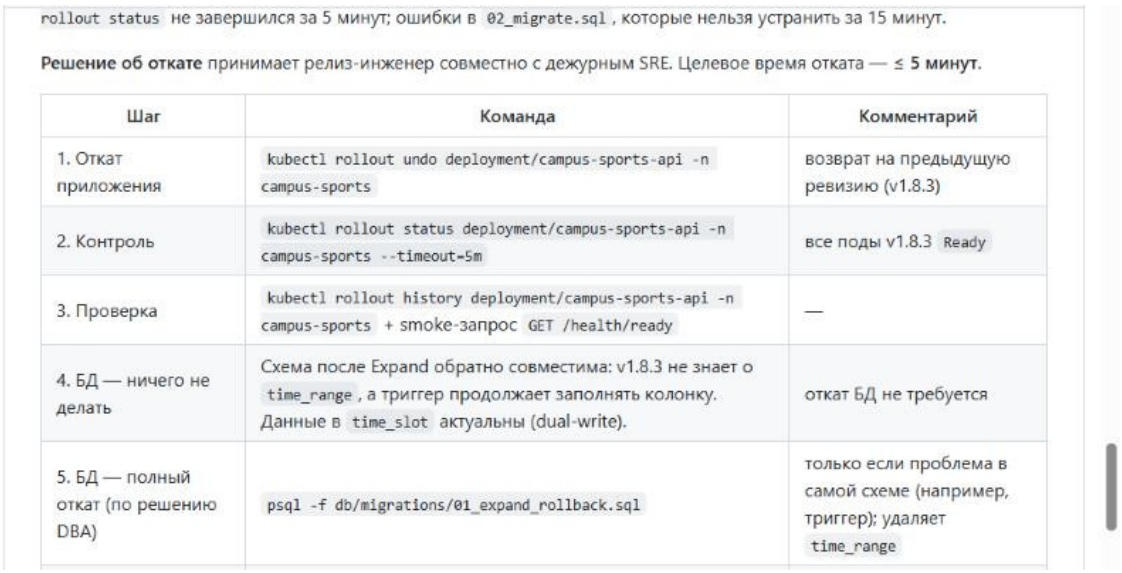
6) 03:25 – фаза Migrate: выполнение 02_migrate.sql и проверка отсутствия NULL, ошибок и расхождений;

7) 03:35 – проверка трейсов по jira.issue = CAMPUS-110, до 04:30 – наблюдение за ошибками, задержкой и блокировками;

8) 04:30 – закрытие CR, запись в CHANGELOG, оповещение в канале релизов.

3.4 План отката

Критерии отката: доля ответов 5xx более 1 % в течение 5 минут, 95-й перцентиль задержки более 800 мс, падение sanity-теста или незавершённый за 5 минут выкат. Действия, целевое время которых не превышает 5 минут, приведены на рисунке 5.



`rollout status` не завершился за 5 минут; ошибки в `02_migrate.sql`, которые нельзя устранить за 15 минут.

Решение об откате принимает релиз-инженер совместно с дежурным SRE. Целевое время отката — ≤ **5 минут**.

Шаг	Команда	Комментарий
1. Откат приложения	`kubectl rollout undo deployment/campus-sports-api -n campus-sports`	возврат на предыдущую ревизию (v1.8.3)
2. Контроль	`kubectl rollout status deployment/campus-sports-api -n campus-sports --timeout=5m`	все поды v1.8.3 `Ready`
3. Проверка	`kubectl rollout history deployment/campus-sports-api -n campus-sports` + smoke-запрос `GET /health/ready`	—
4. БД — ничего не делать	Схема после Expand обратно совместима: v1.8.3 не знает о `time_range`, а триггер продолжает заполнять колонку. Данные в `time_slot` актуальны (dual-write).	откат БД не требуется
5. БД — полный откат (по решению DBA)	`psql -f db/migrations/01_expand_rollback.sql`	только если проблема в самой схеме (например, триггер); удаляет `time_range`

Рисунок 5 – План отката в CR: kubectl rollout undo, контроль и действия с БД

Откат БД не требуется: после фазы Expand схема обратно совместима – версия 1.8.3 не знает о колонке time_range, а time_slot продолжает заполняться. При проблеме в самой схеме DBA выполняет 01_expand_rollback.sql. Точка невозврата наступает только в релизе 1.10.0 при выполнении 03_contract.sql, который оформляется отдельным CR.

4 ПРАВИЛА ЗАЩИТЫ ВЕТКИ MAIN

Конфигурация защиты ветки main описана в docs/branch-protection.md и продублирована в виде JSON для применения через GitHub API [3]. Настройки приведены в таблице 3.

Таблица 3 – Правила защиты ветки main

Настройка	Значение
Require a pull request before merging	да – прямой push в main запрещён
Required approvals	2 одобрения
Require review from Code Owners	да – изменения db/migrations одобряет DBA
Dismiss stale approvals, require approval of the most recent push	да
Require status checks to pass	commit-lint, build-and-test, db-migrations
Require branches to be up to date	да
Require conversation resolution	да
Require signed commits, linear history	да
Do not allow bypassing (включая администраторов)	да
Allow force pushes, deletions	нет

Источник: составлено автором

Обязательные проверки CI определены в workflow homework-5 CI: commit-lint проверяет, что каждый коммит и заголовок PR начинаются с [CAMPUS-NNN]; build-and-test собирает решение и запускает все тесты xUnit; db-migrations поднимает PostgreSQL 16 и последовательно применяет миграции, проверяя идемпотентность и откат первой фазы. Для тегов версий действует защита тегов, а развёртывание в production требует ручного одобрения тимлида и SRE.

Все три обязательные проверки выполнены успешно на последнем коммите ветки main (рисунок 6).

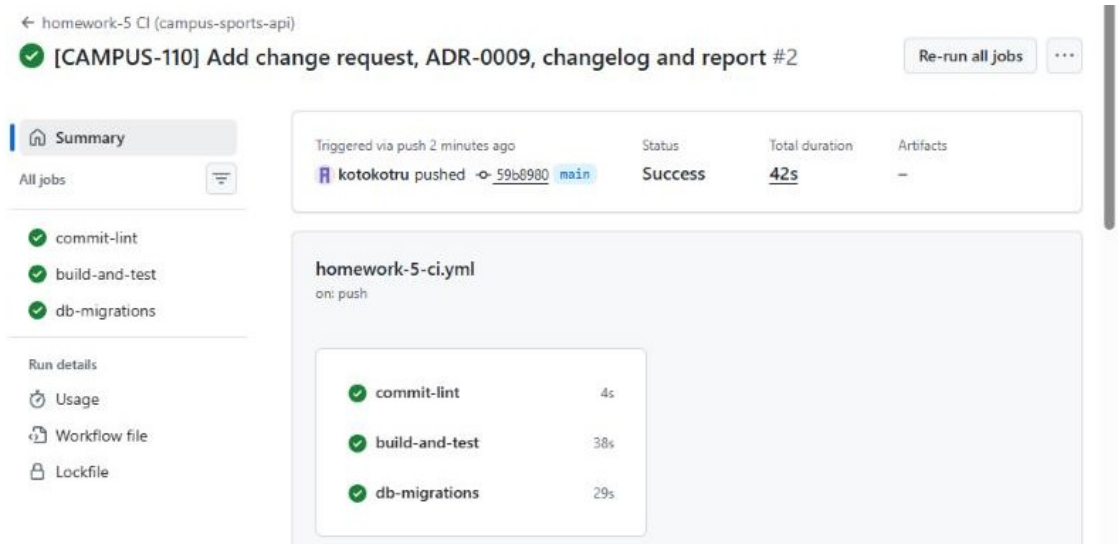


Рисунок 6 – Проверки commit-lint, build-and-test и db-migrations в GitHub Actions

5 МИГРАЦИЯ БАЗЫ ДАННЫХ ПО ПАТТЕРНУ EXPAND AND CONTRACT

5.1 Общая схема миграции

Миграция разделена на три фазы и два релиза, чтобы со схемой в каждый момент могли работать и старые (1.8), и новые (1.9) экземпляры приложения без простоя [4, 8]. Схема фаз приведена на рисунке 7; она выполнена в draw.io, исходный файл expand_contract.drawio хранится в папке report/schemes репозитория.

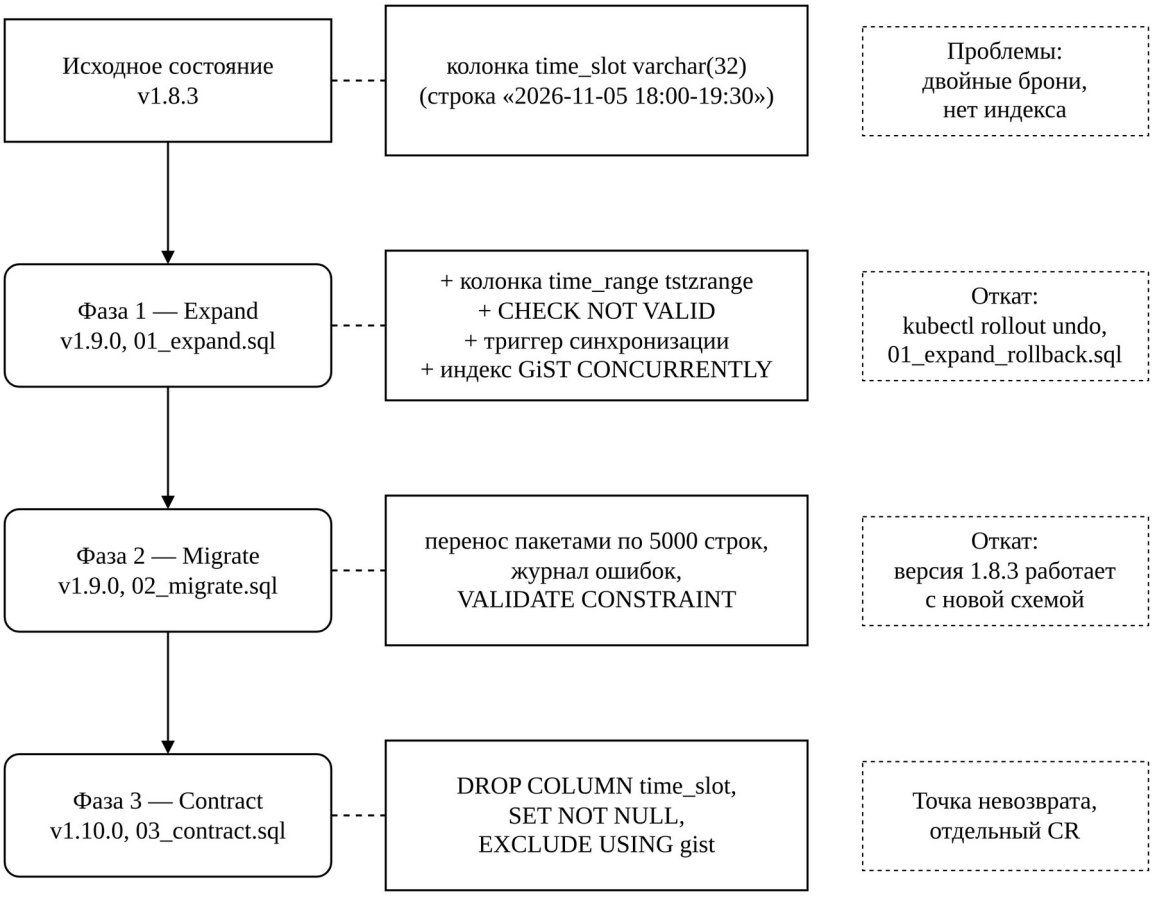


Рисунок 7 – Фазы миграции Expand and Contract

5.2 Фаза Expand

Скрипт 01_expand.sql (версия 1.9.0) добавляет колонку time_range tstzrange без значения по умолчанию – это изменение только каталога, оно выполняется за миллисекунды. Ограничение CHECK на корректность интервала создаётся с параметром NOT VALID, без сканирования таблицы. Создаются функции разбора и формирования строки старого формата и триггер синхронизации, который во время rolling update заполняет новую колонку для записей от подов версии 1.8. Индекс GiST по (facility_id, time_range) строится с параметром CONCURRENTLY, установлен lock_timeout 3 с (рисунок 8).

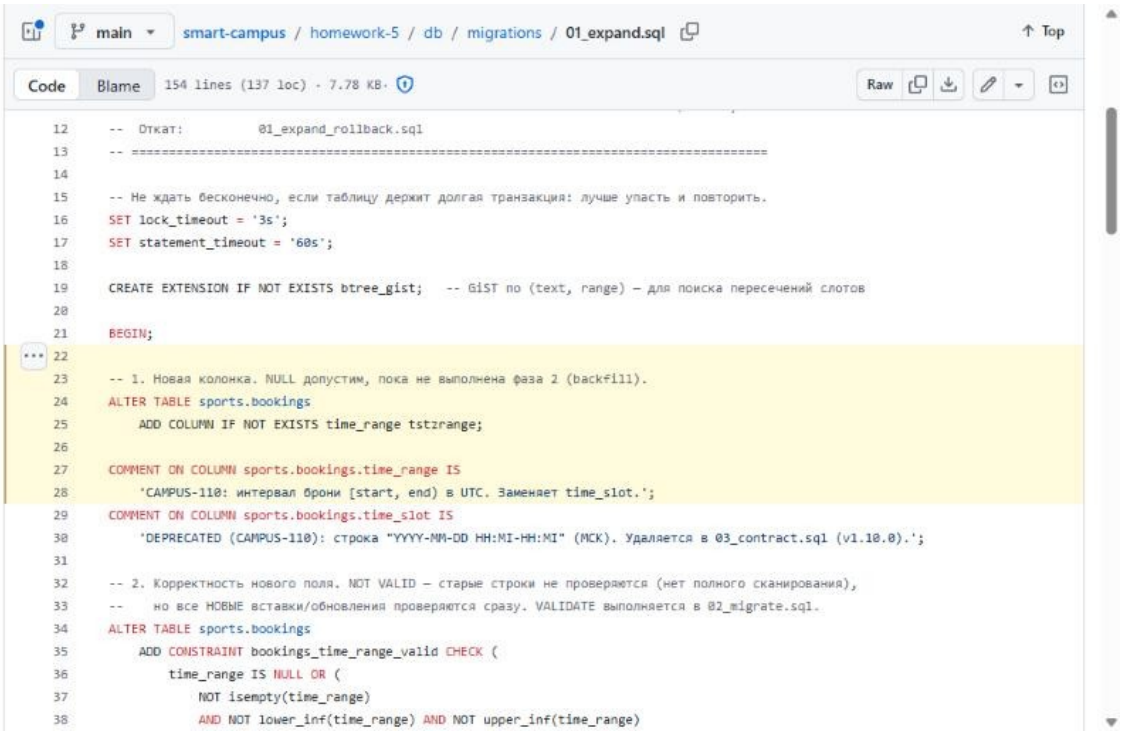


Рисунок 8 – Добавление колонки time_range и ограничения CHECK NOT VALID

Код на C# одновременно пишет данные в старую и новую структуру (dual-write) в одной команде INSERT; запись в старую колонку управляется флагом конфигурации (рисунок 9).

main smart-campus / homework-5 / src / Campus.Sports.Api / Data / NpgsqlBookingRepository.cs ↑ Top

Code Blame 135 lines (120 loc) · 6.35 KB · Raw

```
33      public async Task CreateAsync(SportsBooking booking, CancellationToken ct)
61          var writeLegacy = migration.CurrentValue.WriteLegacyTimeSlot;
62
63          await using (var insert = new NpgsqlCommand("""
64              INSERT INTO sports.bookings
65                  (id, facility_id, student_id, time_range, time_slot, status, is_sanity_test, created_at)
66              VALUES
67                  (@id, @f, @s, @r, @slot, @status, @sanity, @created)
68              """, conn, tx))
69          {
70              insert.Parameters.AddWithValue("id", booking.Id);
71              insert.Parameters.AddWithValue("f", booking.FacilityId);
72              insert.Parameters.AddWithValue("s", booking.StudentId);
73              insert.Parameters.Add(RangeParameter("r", booking.TimeRange));              // НОВАЯ структура
74              insert.Parameters.Add(new NpgsqlParameter("slot", NpgsqlDbType.Varchar)     // СТАРАЯ структура
75              {
76                  Value = writeLegacy ? booking.LegacyTimeSlot : DBNull.Value
77              });
78              insert.Parameters.AddWithValue("status", booking.Status.ToString());
79              insert.Parameters.AddWithValue("sanity", booking.IsSanityTest);
80              insert.Parameters.AddWithValue("created", booking.CreatedAt.UtcDateTime);
81              await insert.ExecuteNonQueryAsync(ct);
82          }
83          // ------------------------------------------------------------------------------
```

Рисунок 9 – Dual-write: одна команда INSERT пишет новую и старую структуры

5.3 Фаза Migrate

Процедура sports.backfill_time_range из скрипта 02_migrate.sql переносит исторические данные пакетами по 5000 строк; каждый пакет – отдельная короткая транзакция, строки выбираются с FOR UPDATE SKIP LOCKED, между пакетами выдерживается пауза. Нераспознанные строки записываются в журнал sports.bookings_migration_errors. После переноса выполняется проверка отсутствия NULL, ошибок и расхождений, затем создаётся ограничение NOT NULL в форме CHECK NOT VALID и выполняется VALIDATE CONSTRAINT с блокировкой, не мешающей записи. Скрипт идемпотентен. Журнал выполнения в CI показан на рисунке 10.

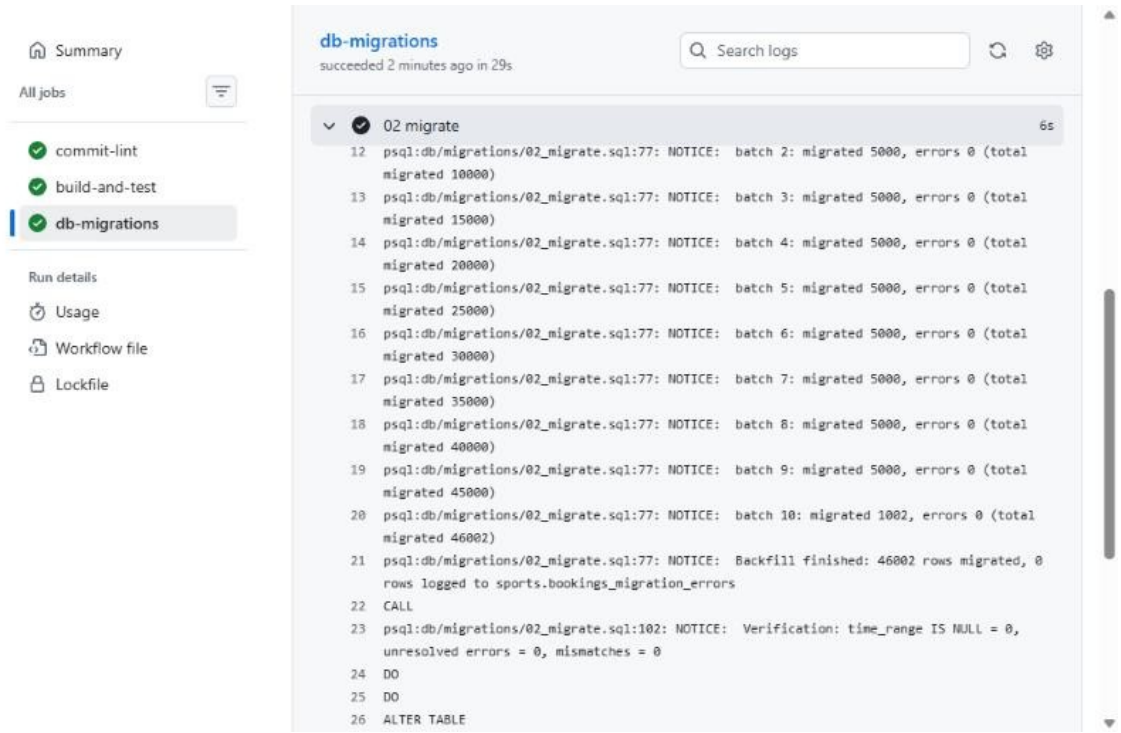


Рисунок 10 – Выполнение 02_migrate.sql в CI: 10 пакетов, 46 002 строки

5.4 Фаза Contract

Скрипт 03_contract.sql выполняется в версии 1.10.0 после перевода всех клиентов на новый формат и 14 дней без запросов в старом формате. Он

проверяет предусловия, удаляет триггер, устанавливает NOT NULL без сканирования благодаря действующему CHECK, удаляет колонку time_slot и служебные функции и добавляет ограничение EXCLUDE USING gist (facility_id WITH =, time_range WITH &&).

5.5 Проверка миграций

Все скрипты выполнены на PostgreSQL 16 на демонстрационном наборе из 46 002 броней; результаты приведены в таблице 4. Тот же сценарий автоматически повторяет CI.

Таблица 4 – Результаты прогона миграций

Шаг	Результат
01_expand.sql	0,35 с; запись от «старого» клиента без time_range – триггер заполнил интервал
02_migrate.sql (с ошибочной строкой)	46 002 строки перенесены за 10 пакетов; строка «завтра вечером» попала в журнал ошибок, скрипт остановился до VALIDATE
02_migrate.sql (после исправления)	NULL – 0, ошибок – 0, расхождений – 0; ограничения действуют; повторный запуск без изменений
03_contract.sql	time_slot удалена, time_range NOT NULL; пересекающаяся бронь отклонена ограничением исключения
01_expand_rollback.sql	схема возвращена к версии 1.8, данные (46 002 строки) сохранены

Источник: составлено автором

6 PRODUCTION SANITY-ТЕСТ

Тест CreateBooking_InTimeRangeFormat_ThenCleanup класса ProductionSanityTests (xUnit, категория Sanity) выполняет точечную проверку боевого сценария сразу после релиза – создание тестовой брони:

1) создаётся бронь служебной площадки sanity-court-00 в новом формате на 15 минут в ночное время случайного дня через год и более, чтобы не мешать реальным пользователям; заголовок X-Campus-Sanity помечает её как тестовую;

2) проверяется ответ 201 Created, точное совпадение границ интервала и заполнение старого поля timeSlot;

3) бронь читается через GET и сравнивается с ответом создания;

4) повторная бронь того же слота должна вернуть 409 Conflict;

5) очистка в DisposeAsync выполняется всегда, даже при падении проверок: DELETE удаляет тестовую бронь, после чего GET должен вернуть 404 (рисунок 11).

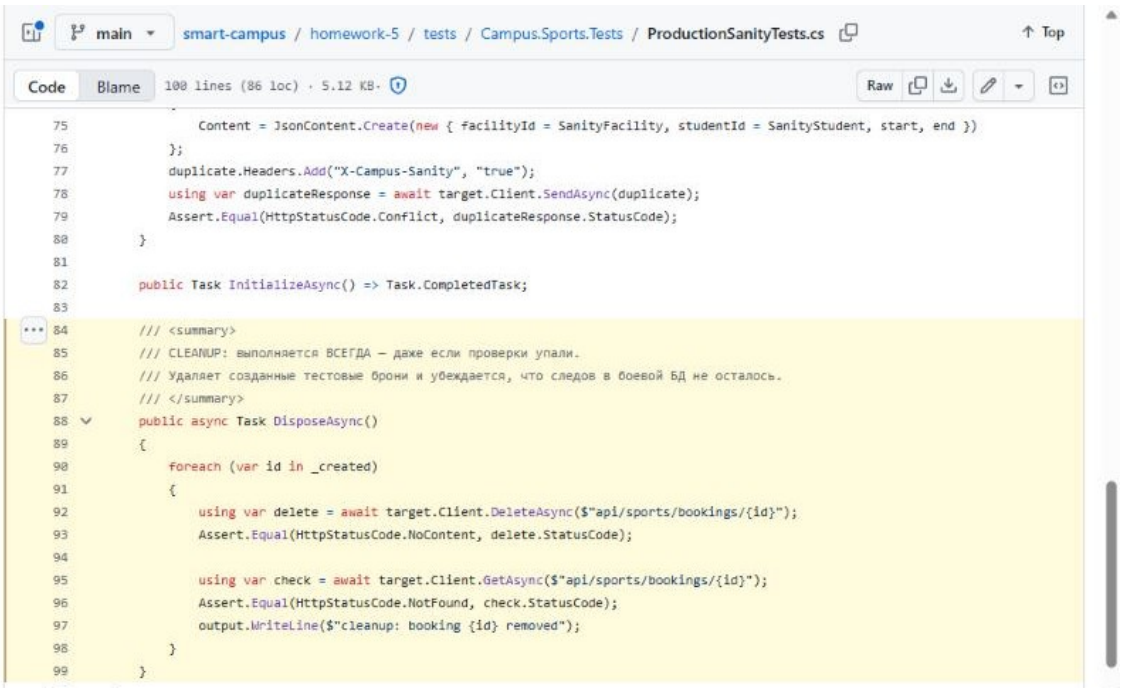


Рисунок 11 – Шаг очистки в DisposeAsync: удаление тестовой брони и проверка 404

После релиза тест запускается с переменной окружения SANITY_BASE_URL (адрес боевого стенда) и фильтром Category=Sanity.

Если переменная не задана, фикстура поднимает сервис внутри процесса тестов, поэтому тот же сценарий выполняется в CI на каждом PR.

На рабочем компьютере пройдено 12 тестов из 12, в том числе sanity-тест (рисунок 12); тот же результат получен в GitHub Actions (рисунок 13).

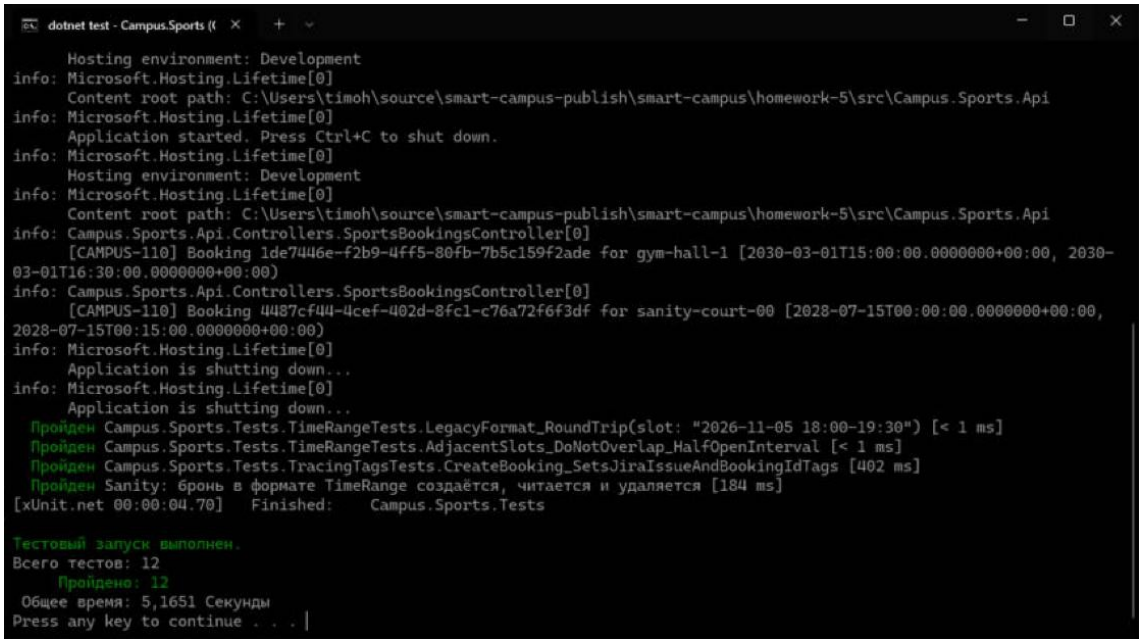


Рисунок 12 – Локальный запуск dotnet test: пройдено 12 тестов из 12

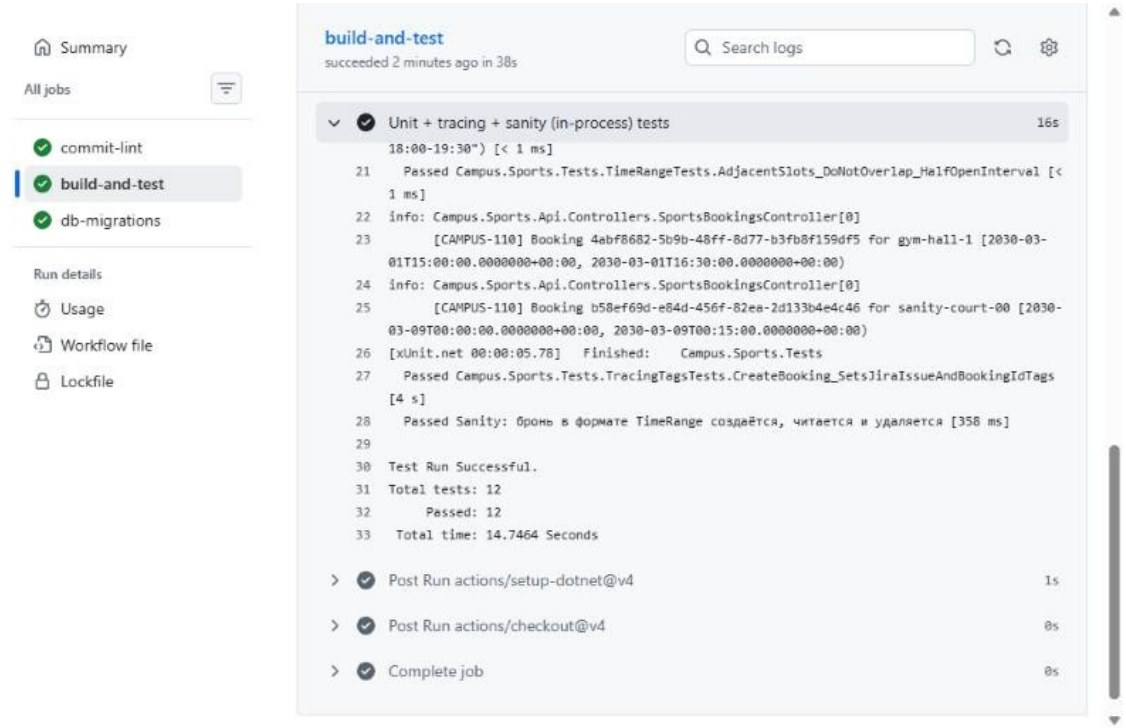
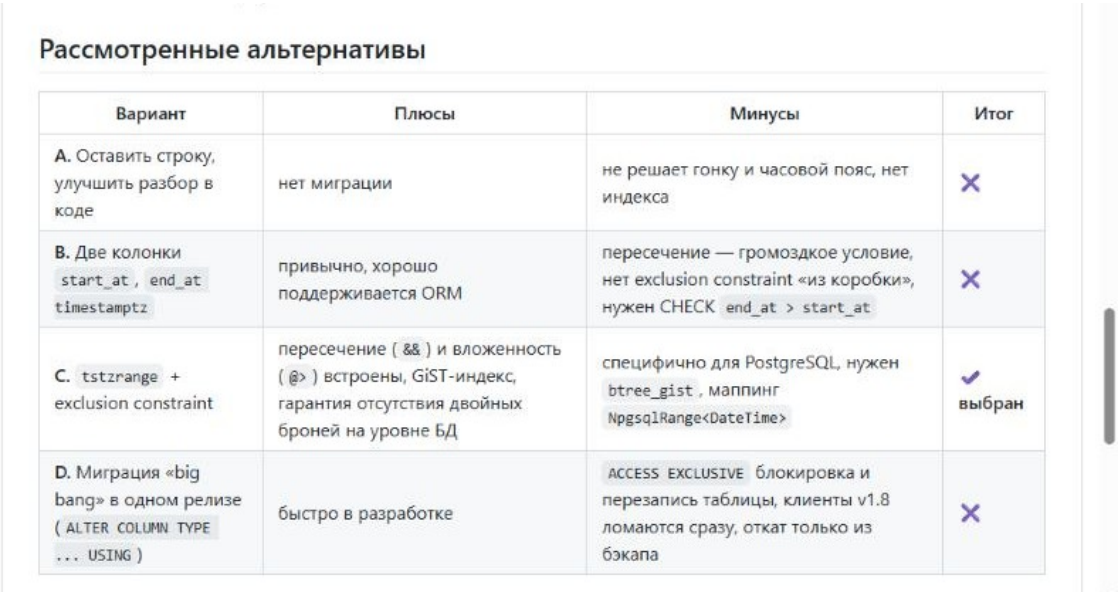


Рисунок 13 – Запуск тестов в GitHub Actions (задание build-and-test)

7 ЭКСПЛУАТАЦИОННАЯ ДОКУМЕНТАЦИЯ

7.1 Архитектурное решение ADR-0009

Документ docs/adr/0009-timeslot-to-timerange.md фиксирует архитектурное решение по варианту в формате ADR [9]: контекст (двойные брони, часовые пояса, два формата строки, отсутствие индексов), принятое решение (TimeRange и tstzrange, ограничение исключения, миграция Expand and Contract в двух релизах, dual-write и триггер), рассмотренные альтернативы (рисунок 14) и последствия.



Рассмотренные альтернативы

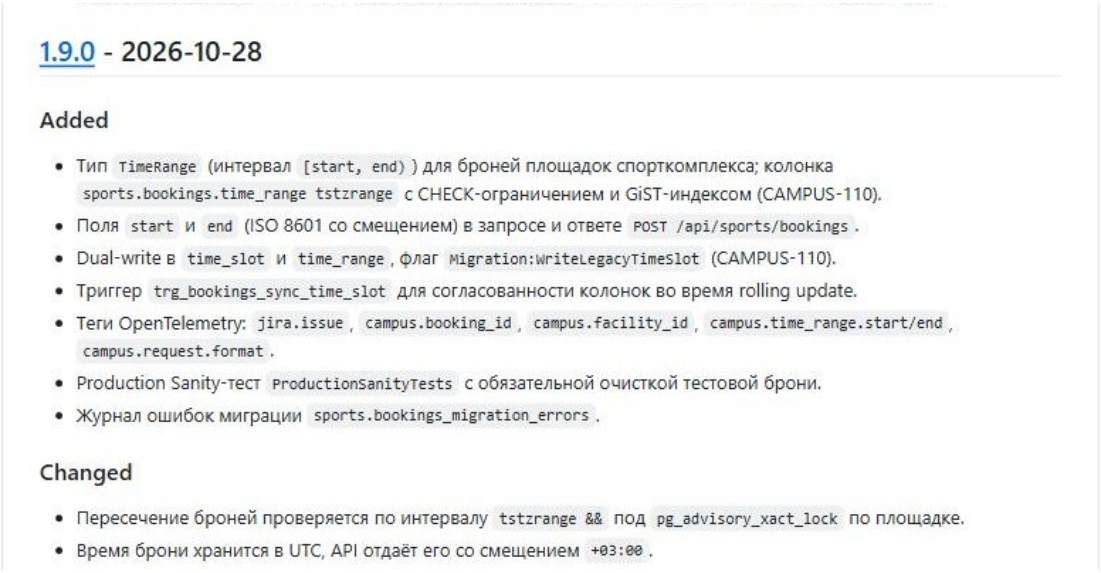
Вариант	Плюсы	Минусы	Итог
A. Оставить строку, улучшить разбор в коде	нет миграции	не решает гонку и часовой пояс, нет индекса	✕
B. Две колонки `start_at`, `end_at` `timestamptz`	привычно, хорошо поддерживается ORM	пересечение — громоздкое условие, нет exclusion constraint «из коробки», нужен CHECK `end_at > start_at`	✕
C. `tstzrange` + exclusion constraint	пересечение (`&&`) и вложенность (`@>`) встроены, GiST-индекс, гарантия отсутствия двойных броней на уровне БД	специфично для PostgreSQL, нужен `btree_gist`, маппинг `NpgsqlRange<DateTime>`	✔ выбран
D. Миграция «big bang» в одном релизе (`ALTER COLUMN TYPE ... USING`)	быстро в разработке	`ACCESS EXCLUSIVE` блокировка и перезапись таблицы, клиенты v1.8 ломаются сразу, откат только из бэкапа	✕

Рисунок 14 – Раздел «Рассмотренные альтернативы» в ADR-0009

Отрицательные последствия: временное усложнение (две колонки, триггер, флаг) до версии 1.10.0 и точка невозврата при удалении колонки, поэтому фаза Contract вынесена в отдельный CR.

7.2 Журнал изменений

CHANGELOG.md ведётся по стандарту Keep a Changelog [10] и правилам семантического версионирования. Запись [1.9.0] - 2026-10-28 содержит разделы Added (TimeRange, колонка time_range, dual-write, теги OpenTelemetry, sanity-тест), Changed, Deprecated (поле timeSlot), Fixed (двойное бронирование, сдвиг времени на 3 часа, формат даты) и Breaking Changes (в 1.9.0 их нет, релиз обратно совместим). В разделе Unreleased анонсированы изменения версии 1.10.0 (рисунок 15).



1.9.0 - 2026-10-28

Added

- Тип `TimeRange` (интервал `[start, end)`) для броней площадок спорткомплекса; колонка `sports.bookings.time_range tstzrange` с CHECK-ограничением и GiST-индексом (CAMPUS-110).
- Поля `start` и `end` (ISO 8601 со смещением) в запросе и ответе `POST /api/sports/bookings`.
- Dual-write в `time_slot` и `time_range`, флаг `Migration:WriteLegacyTimeSlot` (CAMPUS-110).
- Триггер `trg_bookings_sync_time_slot` для согласованности колонок во время rolling update.
- Теги OpenTelemetry: `jira.issue`, `campus.booking_id`, `campus.facility_id`, `campus.time_range.start/end`, `campus.request.format`.
- Production Sanity-тест `ProductionSanityTests` с обязательной очисткой тестовой брони.
- Журнал ошибок миграции `sports.bookings_migration_errors`.

Changed

- Пересечение броней проверяется по интервалу `tstzrange &&` под `pg_advisory_xact_lock` по площадке.
- Время брони хранится в UTC, API отдаёт его со смещением `+03:00`.

Рисунок 15 – Запись о релизе 1.9.0 в CHANGELOG.md

ЗАКЛЮЧЕНИЕ

В ходе практической работы для модуля «Спортивный комплекс» ИС «Умный кампус» (вариант 10, задача CAMPUS-110) спроектирован безопасный релиз изменения формата TimeSlot на TimeRange.

Реализован единый цифровой след: каждый запрос к API несёт теги jira.issue и campus.booking_id, а цепочка «задача – коммит – CI – релиз – трейс – миграция» прослеживается от начала до конца. Оформлен Change Request на релиз 1.9.0 с обоснованным окном обслуживания, пошаговым планом работ и планом отката через kubectl rollout undo, описаны правила защиты ветки main с обязательными проверками CI.

Миграция базы данных выполнена по паттерну Expand and Contract в три фазы: добавление новой структуры с dual-write в коде, пакетный перенос исторических данных с контролем и удаление старой структуры отдельным релизом. Все скрипты проверены на PostgreSQL 16: миграция проходит без блокировок записи, идемпотентна, обнаруживает некорректные данные и откатывается. Итоговая схема физически исключает двойное бронирование площадок.

Написан production sanity-тест с обязательной очисткой тестовых данных, составлены ADR-0009 и журнал изменений по стандарту Keep a Changelog. Работа позволила закрепить навыки управления изменениями, безопасной эволюции схемы БД без простоя и ведения документации по принципу Docs-as-Code. В дальнейшем после перевода всех клиентов на новый формат необходимо оформить отдельный CR на фазу Contract в версии 1.10.0.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Instrumentation. OpenTelemetry .NET // OpenTelemetry : сайт. – URL: https://opentelemetry.io/docs/languages/dotnet/instrumentation/ (дата обращения: 22.10.2026). – Текст : электронный.

2. Attribute Naming. Semantic Conventions // OpenTelemetry : сайт. – URL: https://opentelemetry.io/docs/specs/semconv/general/naming/ (дата обращения: 22.10.2026). – Текст : электронный.

3. About protected branches // GitHub Docs : сайт. – URL: https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches (дата обращения: 22.10.2026). – Текст : электронный.

4. ALTER TABLE. PostgreSQL 16 Documentation // PostgreSQL : сайт. – URL: https://www.postgresql.org/docs/16/sql-altertable.html (дата обращения: 22.10.2026). – Текст : электронный.

5. Range Types. PostgreSQL 16 Documentation // PostgreSQL : сайт. – URL: https://www.postgresql.org/docs/16/rangetypes.html (дата обращения: 22.10.2026). – Текст : электронный.

6. Deployments. Rolling Back a Deployment // Kubernetes Documentation : сайт. – URL: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/ (дата обращения: 22.10.2026). – Текст : электронный.

7. ITIL 4. Change Enablement Practice // AXELOS : сайт. – URL: https://www.axelos.com/certifications/itil-service-management (дата обращения: 22.10.2026). – Текст : электронный.

8. Sadalage, P. Parallel Change / P. Sadalage // martinfowler.com : сайт. – URL: https://martinfowler.com/bliki/ParallelChange.html (дата обращения: 22.10.2026). – Текст : электронный.

9. Nygard, M. Documenting Architecture Decisions / M. Nygard // Cognitect : сайт. – URL: https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions (дата обращения: 22.10.2026). – Текст : электронный.

10. Keep a Changelog 1.1.0 // keepachangelog.com : сайт. – URL: https://keepachangelog.com/ru/1.1.0/ (дата обращения: 22.10.2026). – Текст : электронный.